```
        # Validate the Venue was created
        venue = Venue.objects.first()
        self.assertEqual(data['name'], venue.name)
        self.assertEqual(data['description'], venue.description)
        self.assertTrue(
            self.owner.userprofile.venues_controlled.filter(
                id=venue.id).exists()
        )
```

Fetch the newly created Venue from the database.

Validate the Venue object's attributes against the expected data.

Verify that the owner's UserProfile indicates they own the new Venue.

The `edit_venue()` view is used for both creating and editing a `Venue`, so now, it is time to add code testing `Venue` object edits. This situation is similar to creation, but instead, the URL contains the `Venue` object's ID and the data is set to change a field. The following listing shows the code that needs to be added to the test.

Listing 8.8 Test editing a Venue

```
# bands/tests.py
...
class TestBands(TestCase):
    ...
    def test_edit_venue(self):
        ...
        # Now edit that Venue
        url = f"/bands/edit_venue/{venue.id}/"
        data['name'] = "Edited Name"
        response = self.client.post(url, data)

        self.assertEqual(302, response.status_code)
        venue = Venue.objects.first()
        self.assertEqual(data['name'], venue.name)
```

Change the URL to point to the Venue getting edited.

Set a new value for the Venue's name.

Post the change request.

Check that the Venue content has changed.

WARNING Remember, `Model` objects are proxies; they won't update automatically. When your view changes the database, you need to refetch the `Model` object.

The last thing to test is the permission check. Re-attempt the edit with another user, and verify their attempt to edit gets denied. Listing 8.9 shows this new code; it is similar to the previous code, except this time the logged-in user is *member*.

Listing 8.9 Test Venue editing permissions

```
# bands/tests.py
...
class TestBands(TestCase):
    ...
    def test_edit_venue(self):
        ...
        # Verify that a non-owner can't edit
        self.client.login(username="member", password=self.PASSWORD)
        response = self.client.post(url, data)
        self.assertEqual(404, response.status_code)
```

Edit requests from users without permission cause an HTTP 404 error.